

Credit Count — Technical Design Document

Author	Javier Arias	Date	10 September 2026
Version	1.0 — draft for approval, build not started	Source	Credit Count — Statement of Work, Candidate Task, Koin Limited, July 2026
History	1.0 (10 Sep 2026) initial design for approval. An "as built" revision will follow the build and note any deviation.		

1. Summary and design principle

Credit Count is a small multi-user web app where rollercoaster enthusiasts log rides against a shared catalogue and see their **credit count** (unique coasters ridden), total rides and stats, with an **opt-in** public leaderboard. Admins maintain the catalogue; visitors see only the leaderboard and sign-up. One principle drives the design: **the database is the security boundary**. Every privacy and role rule in the SOW is enforced with Postgres Row Level Security (RLS), grants and `security definer` functions, so it holds identically for the app, for direct API calls and for anything built later (FR6, FR8, FR9; AC2, AC4). The Next.js layer adds UX, validation and friendly errors; it is never the only line of defence.

2. Architecture

Browser → Next.js 16 (App Router) on Vercel → Supabase. React Server Components read data; Server Actions write; `proxy.ts` (Next 16's middleware) refreshes the session cookie and redirects unauthenticated users. Supabase provides Auth (email + password), Postgres with RLS, and the few pieces of server-side logic that need elevated rights (SQL functions). There is no custom API layer and **no service-role key anywhere**: the app uses the publishable (anon) key plus the user's session via `@supabase/ssr`; RLS decides what each request may see.

Layer	Choice	Reason
Front + server	Next.js 16.3, React 19, TypeScript, Tailwind, shadcn/ui, Zod	Required stack; RSC + Server Actions keep the app small; Zod validates every mutation.
Data + auth	Supabase Auth, Postgres 17, RLS, SQL functions	Required stack; rules live once, in the database.
Hosting	Vercel Hobby (prod = <code>main</code> , previews per PR); Supabase Free in <code>us-east-1</code> ; Vercel functions in <code>iad1</code>	Free tiers; app and database in the same region.
Tests	Vitest for stats logic; <code>scripts/rls-check.ts</code> proves RLS through the API	AC2 and AC4 are demonstrated, not assumed.

3. Data model

Table	Columns and constraints
profiles	<code>id</code> uuid PK = <code>auth.users.id</code> · <code>display_name</code> text (2–40 chars) · <code>role</code> text in (<code>enthusiast</code> , <code>admin</code>), default <code>enthusiast</code> · <code>show_on_leaderboard</code> bool default false · <code>created_at</code> . Created by a trigger on <code>auth.users</code> insert, taking <code>display_name</code> from sign-up metadata.
coasters	<code>id</code> uuid PK · <code>name</code> · <code>park</code> · <code>country</code> · <code>manufacturer</code> · <code>type</code> text in (<code>steel</code> , <code>wooden</code> , <code>hybrid</code>) · <code>rcdb_id</code> int null (reserved for the future RCDB sync) · <code>created_at</code> , <code>updated_at</code> . Unique index on (<code>lower(name)</code>), (<code>lower(park)</code>) blocks duplicates at the source.
rides	<code>id</code> uuid PK · <code>user_id</code> uuid → profiles (cascade) · <code>coaster_id</code> uuid → coasters (restrict) · <code>ridden_on</code> date default today, not in the future · <code>note</code> text ≤ 280 · <code>created_at</code> . Indexes on (<code>user_id</code>) and (<code>user_id</code> , <code>coaster_id</code>).

Credits and stats are derived, never stored. Credits = `count(distinct coaster_id)` per user; rides = `count(*)`. The dashboard reads the user's rides joined to coasters (a few hundred rows at most) and a pure `computeStats()` function produces credits by country, manufacturer and type, plus the most-ridden coaster. No denormalised counters can drift, and every Server Action calls `revalidatePath`, so dashboard, history and leaderboard reflect changes immediately (FR3–FR5, FR7).

4. Access control

Object	Visitor (anon)	Enthusiast (authenticated)	Admin
profiles	none	SELECT own row; UPDATE own row with a column grant limited to <code>display_name</code>, <code>show_on_leaderboard</code> (<code>role</code> is not writable through the API)	same as enthusiast

Object	Visitor (anon)	Enthusiast (authenticated)	Admin
coasters	none	SELECT	SELECT + INSERT/UPDATE/DELETE where (select is_admin())
rides	none	ALL where user_id = auth.uid() (USING and WITH CHECK)	no extra policy — admins cannot read other users' rides (SOW §3)
get_leaderboard()	EXECUTE	EXECUTE	EXECUTE
merge_coaster(from, into)	—	raises unless admin	reassigns rides to the surviving coaster, deletes the duplicate

- `is_admin()` is security definer, stable, with a fixed `search_path`, reading `profiles.role`: policies never recurse and a role change applies immediately, no JWT refresh. Admin is granted manually by SQL ("no self-serve admin sign-up").
- `get_leaderboard()` is the **only** path by which a visitor touches data. It returns `display_name`, `credits` and `rides` (tie-break) for profiles with `show_on_leaderboard = true`, ordered by credits; never user ids or coaster ids, so it cannot reveal what anyone has ridden (FR7). The page renders dynamically, so opting out takes effect on the next request.
- Default table privileges for `anon` and `authenticated` are revoked and re-granted only as above; RLS is enabled on all three tables. Server Actions repeat the checks for friendly errors, but the database refuses the write regardless.
- Only the publishable key ships to the browser (designed to be public; RLS protects the data). The service-role key is never used: the catalogue is seeded by a SQL migration and test accounts are created through normal sign-up.

5. Application design

Route	Access	Content
/	public	Leaderboard (rank, display name, credits) with sign-in / sign-up calls to action
/signup, /login	public	Email, password, display name at sign-up
/dashboard	user	Headline credits , total rides, stats by country / manufacturer / type, most-ridden coaster, Quick log
/rides	user	Ride history: edit date and note, delete with confirmation
/coasters	user	Browse and search the catalogue; "Log ride" on each row
/settings	user	Display name and the leaderboard toggle, with a plain-language note on what becomes public
/admin/coasters	admin	Add, edit, delete, and "merge into" for duplicates

Logging a ride in three interactions (FR2): the dashboard has an always-visible Quick log card: (1) type in the coaster search box, (2) pick the coaster, (3) press "Log ride"; the date defaults to today and the note is optional. Mobile-first: everything works at 400 px width with no horizontal scroll.

Sign-up without email confirmation (decision). The SOW asks for email + password sign-up and leaves richer email flows out of scope. Supabase's built-in email service on the free tier allows only a handful of messages per hour, which would stop reviewers from creating accounts on the spot. Confirmation is therefore **disabled for v1** and recorded as a known trade-off; v2 adds custom SMTP and turns it back on.

6. Requirements traceability

SOW requirement	How the design meets it
FR1 visitors see only leaderboard and sign-up	<code>anon</code> has no table privileges; only <code>get_leaderboard()</code> is executable.
FR2 log a ride in ≤ 3 interactions	Quick log card on the dashboard (§5).
FR3–FR5 rides vs credits; stats always current	One row per ride; credits = distinct coasters as the headline; derived on read + <code>revalidatePath</code> .
FR6, FR9 ride history private; edit/delete own rides only	RLS on <code>rides</code> with USING and WITH CHECK on <code>user_id = auth.uid()</code> ; proven by <code>rls-check.ts</code> .
FR7 leaderboard opt-in, name + credits only, opt-out immediate	<code>get_leaderboard()</code> filters on the flag and exposes two fields; dynamic page.
FR8 only admins change the catalogue, at the database layer	<code>is_admin()</code> policies on <code>coasters</code> ; proven by <code>rls-check.ts</code> .
AC1–AC4	Walked through on the deployed app before submission and recorded in the README; AC2 and AC4 also by script.

SOW requirement	How the design meets it
AC5, AC6 no secrets; TDD matches the build	<code>.env*.local</code> ignored, <code>.env.example</code> committed, publishable key only, key grep before push; this document lives in the repo and gets an "as built" revision with deviations in §7.

7. Assumptions, open questions and deviations

Questions sent to Koin on 10 September 2026, each with the default applied if unanswered: (1) admin is a flag on a normal account, so admins may also log rides; (2) removing a duplicate that has rides uses **merge**, so nobody loses credits; (3) opted-in users with zero credits appear, ranked by credits, then rides, then name; (4) email confirmation disabled for v1 (§5); (5) repository is public.

Further assumptions: display names are not unique (SOW §9), so two users can share a name and one could imitate another on the leaderboard; v1 accepts this, v2 adds a unique handle. The catalogue is seeded with about 40 real coasters across ten countries, nine manufacturers and three types; `rcdb_id` is reserved so a future RCDB sync can match rows. English-only interface. **Deviations from the SOW: none planned;** any deviation introduced during the build will be recorded here with its reason.

8. Risks and free-tier limits

Risk / limit	Handling
Privacy leak between users (the SOW's most serious defect)	RLS with USING + WITH CHECK, admins without a <code>rides</code> policy, leaderboard only through a function, <code>rls-check.ts</code> run against production before submission.
Catalogue data quality	Unique index on name + park; admin edit and merge; seed reviewed by hand.
Supabase Free: project pauses after 7 days of inactivity ; 500 MB database; 50k MAU; 2 active projects; built-in email ~2 messages/hour	Daily Vercel cron pings the database so the demo stays live; email confirmation off (§5). At real scale the first upgrades are custom SMTP and the Pro plan.
Vercel Hobby: non-commercial only; 100 GB bandwidth; 10 s default function timeout; cron once a day	Fine for v1; every page is a handful of small queries.
Time-box (5–8 h)	Build order in §9; the merge action is the first cut if time runs short, recorded in §7.

9. Verification, delivery and operations

Build order: migrations (schema, RLS, functions, seed) → auth → dashboard and Quick log → ride history → leaderboard → catalogue, settings, admin → polish and mobile pass → tests → deploy → demo data → TDD "as built". **Migrations** are SQL files under `supabase/migrations`, applied in order and committed; the seed is idempotent. **Environment:** `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY`, set in Vercel and in a local `.env.local`. **Deploy:** GitHub → Vercel Git integration; every push to `main` is production. **Tests:** Vitest for `computeStats()`; `rls-check.ts` signs in as two throwaway users with the publishable key and asserts that cross-user reads return nothing, cross-user updates and deletes touch zero rows, an enthusiast cannot write to `coasters`, and a visitor cannot read `coasters` or `rides`. **Handover:** live Vercel URL, one enthusiast and one admin test account (credentials sent separately, never in the repository), public repository with README, this TDD and `docs/AI_WORKFLOW.md` on how the build was directed with Claude Code.

10. Out of scope and next steps

Out of scope per SOW: live RCDB integration, native apps, custom password-reset flows, payments. Suggested v2, in order of value: RCDB sync job using `rcdb_id`; unique handles; custom SMTP with email confirmation; trigram search index and pagination as the catalogue grows; per-park stats and badges; ride photos in Supabase Storage with per-user policies; CSV export; Playwright end-to-end tests; error monitoring.